Physical AI 応用編 2026　最終課題 提出レポート

KumaWatch Patrol：AI による熊出没リスク巡回ロボット

Risk-Aware Robotic Patrol for Bear Encounters

提出者　（氏名）　　提出日　2026年11月
コード　github.com/Global-jogasaki/KumaRobot　　動画　（URL）

> 熊の出没地域では、限られた人員ですべての地点を巡回できない。本作品は、熊出没予測の研究 「KumaWatch」の予測 AI（TTM）が選んだ地点を、ROS 2 と Gazebo 上の移動ロボットが Navigation2 で自律巡回し、カメラ画像の AI 判定（YOLOv8）で熊を見つけると 安全な距離で停止・記録するシステムである。AI を「予測」と「現場確認」の 2 か所で使い、 予測結果をロボットの行動につなげた。同じ 3 地点の巡回で、固定ルートの熊発見数 1 に対し、AI ルートは 2 だった。

1 課題の背景

近年、熊の人里への出没が増え、自治体は通学路やごみ集積所などの見回りに追われている。しかし人員には限りがあり、 「明日はどこを見に行くべきか」を限られた数だけ選ぶ必要がある。筆者らの研究 KumaWatch では、山形県・秋田県の 公開出没データを 10 km グリッドに集計し、翌日の出没リスクが高いセルを上位 K 個選ぶ問題として、 複数の予測手法を比較してきた。一方、予測はあくまで地図上の数値であり、実際に現場を確認し、熊がいれば安全に 行動する部分は人に委ねられている。本作品では、この「予測から現場確認まで」をロボットで具体化する。

2 目的

1. AI の出没リスク予測に基づいて、巡回可能な数（K=3）の地点と巡回順を自動で決める。
2. Navigation2 で候補地点を自律巡回し、カメラ画像を AI で判定して熊を見つける。
3. 熊を見つけたら追跡せずに停止し、位置・時刻・信頼度を記録する（安全側の行動）。
4. 固定ルートの巡回と同じ条件で比較し、AI で地点を選ぶ効果を示す。

3 システム構成

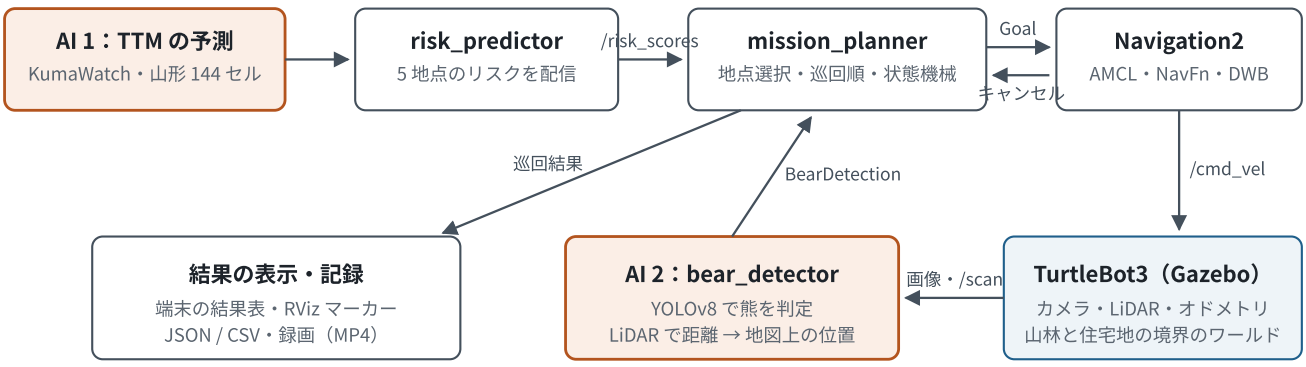


図1　システム構成。橙の枠が AI を使う部分。ノード間は ROS 2 のトピック（リスク・熊検知は独自メッセージ）と Nav2 の `NavigateToPose` アクションでつながる。

ソフトウェアは 5 つの ROS 2 パッケージで構成した（表1）。ワールドと Nav2 用の地図は、同じ定義ファイル（`world_layout.yaml`）から自動生成するので、SLAM で地図を作らなくても座標が一致する。熊とシカは地図に

載せず、 ロボットにとって未知の障害物として LiDAR とカメラで見つける。

ノード	役割
`risk_predictor`	TTM の予測を 5 地点のリスクとして配信し、RViz に棒グラフで表示
`mission_planner`	地点選択・巡回順の決定、Nav2 への移動指示、熊検知時の停止・記録、結果表示（状態機械）
`bear_detector`	カメラ画像を YOLOv8 で判定し、LiDAR の距離から熊の地図上の位置を推定
`image_recorder`	カメラ映像と検出枠入りの映像を状態表示付きで録画、熊検知時の静止画を保存
`bear_proximity_monitor`	熊の既知座標との距離で検知する簡易版（比較・動作確認用）

表1　主なノード。状態機械は RISK_ASSESSMENT → ROUTE_PLANNING → PATROL ⇄ OBSERVE → (EMERGENCY_STOP) → RETURN_TO_BASE → FINISHED と遷移する。

4 実装内容

4.1　AI 1：熊出没リスクの予測（KumaWatch の TTM）

リスク予測には、KumaWatch が公開している IBM Granite TinyTimeMixer（TTM 1536-96-R2）の予測を使った。 TTM は大量の時系列で事前学習された時系列予測の基盤モデルで、KumaWatch では山形県 144 セルの日別出没件数の履歴から、 学習に使っていない 2025 年の各日の出没をゼロショットで予測している。作品の町は架空だが、5 地点のリスクには 実在セルの予測値を割り当てた（表2）。割り当ては次の規則による：2025 年 6 月 7 日に実際に出没があった 2 セルを P2・P4 とし、 Gazebo でもそこに熊を置く。出没がなかったセルからは、TTM 順位が最も高いもの（P3、AI の空振り）、40 位付近（P5）、 100 位付近（P1）を選んだ。リスクは県内 144 セル中の順位を百分位にした値である。

地点	環境	セル	TTM 順位	リスク	実際の出没	Gazebo の熊
P4	ごみ集積所	5_1	1 / 144	1.000	あり	あり
P3	通学路	4_1	2 / 144	0.993	なし（空振り）	なし
P2	林縁	1_10	4 / 144	0.979	あり	あり
P5	河川沿い	6_0	40 / 144	0.727	なし	なし（シカ）
P1	住宅地	1_0	93 / 144	0.357	なし	なし

表2　5 地点に割り当てた TTM の予測（2025 年 6 月 7 日、山形県 10 km グリッド）。

4.2　巡回地点と巡回順の決定

リスクだけでなく、社会的重要度と移動コストを考えた効用で地点を選ぶ。

$$U_i = w_i\, p_i - \lambda\, d_i$$

p_i はリスク、w_i は重要度（通学路 1.2、住宅地 1.1、他 1.0）、d_i は詰所からの距離 [m]、 λ=0.02 である。U_i の上位 3 地点を選び、詰所から回って戻る総移動距離が最小になる順を全順列で求める。 今回は P2（林縁）→ P4（ごみ集積所）→ P3（通学路）となった。比較用に、リスクだけで選ぶ方式と固定ルートも実装した。

4.3　自律巡回と停止

`mission_planner` は Nav2 の `NavigateToPose` アクションで各地点へ移動し、到着後に一定時間周囲を確認する。熊の検知を受け取ると、移動中のゴールをキャンセルしてゼロ速度で停止し、熊の位置・時刻・距離・信頼度を記録する。 熊を追跡せず、止まって記録してから次の地点へ向かう。巡回後は詰所へ戻り、結果を端末・RViz・ファイルに出力する。

4.4　AI 2：カメラ画像からの熊判定（YOLOv8）

COCO で学習済みの YOLOv8 にも「bear」クラスはあるが、Gazebo の熊（基本図形の組み合わせ）は信号機や人と誤認された（図2 左）。 そこで Gazebo の画像で**追加学習（ファインチューニング）**した。カメラと熊・シカをランダムな位置に動かしながら 700 枚を撮影し、熊の 3D 位置を画像へ投影して正解の枠を自動で付けた。木や建物で隠れる視点は除き、熊が写らない画像も 3 割含めた。 学習済みの YOLOv8n を初期値に追加学習した（図2 右）。

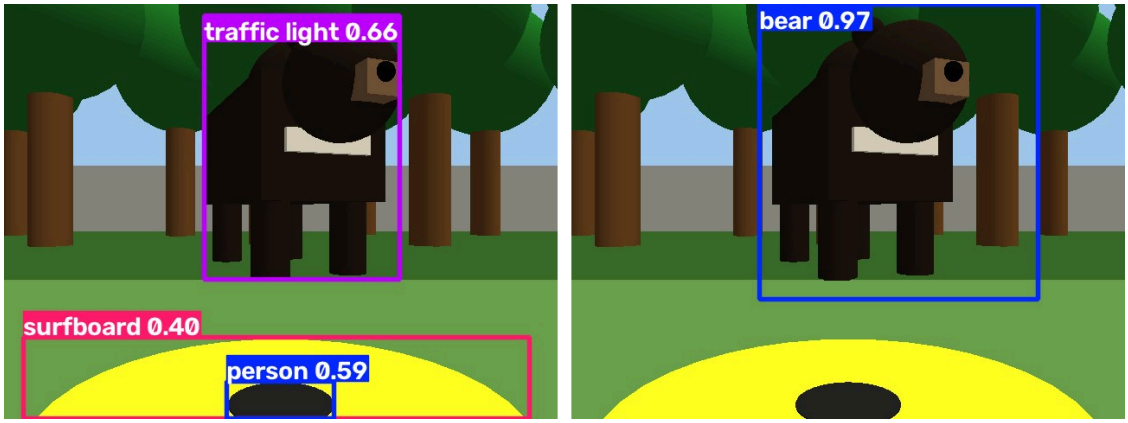


図2　追加学習の前後。左：COCO 学習済みの YOLOv8s は Gazebo の熊を「traffic light」と誤認する。 右：追加学習後の YOLOv8n は「bear」（信頼度 0.97）と判定する。

`bear_detector` は、検出枠の横位置からカメラに対する熊の方位を、その方位の LiDAR の値から距離を求め、 ロボットの自己位置と合わせて熊の地図上の位置を推定する。LiDAR が手前の木を拾った場合は、枠の高さからの推定に切り替える。 誤検出で止まらないよう、3 フレーム連続で検出し、かつ 4 m 以内に近づいたときに停止を指示する。

5 工夫した点

- **研究の予測をそのまま使う。**架空の点数ではなく、KumaWatch の TTM の実際の予測値を使い、実際に出没があったセルに熊を置いた。 AI の空振り（P3）もあえて含め、予測が外れる場合も見えるようにした。
- **危険度だけでなく、重要度と移動距離も考える。**通学路の重みと移動コストを入れた効用で地点を選び、巡回順は総移動距離が最小になるように決めた。
- **ラベル付けを自動化した追加学習。**シミュレータでは熊の位置が分かるので、画像への投影で正解の枠を自動で作れる。 手作業なしで 700 枚の学習データを作り、既存モデルが認識できない対象に対応させた。
- **安全側の行動。**熊を見つけても近づかず、4 m 以内で停止して記録する。連続検出で確定させ、1 フレームの誤検出では止まらないようにした。
- **ワールドと地図を 1 つの定義から生成。**配置を変えても地図が自動で一致する。開けた場所で自己位置推定がずれた問題には、 電柱やベンチを目印として加えて対処した。

6 結果

6.1　固定巡回との比較

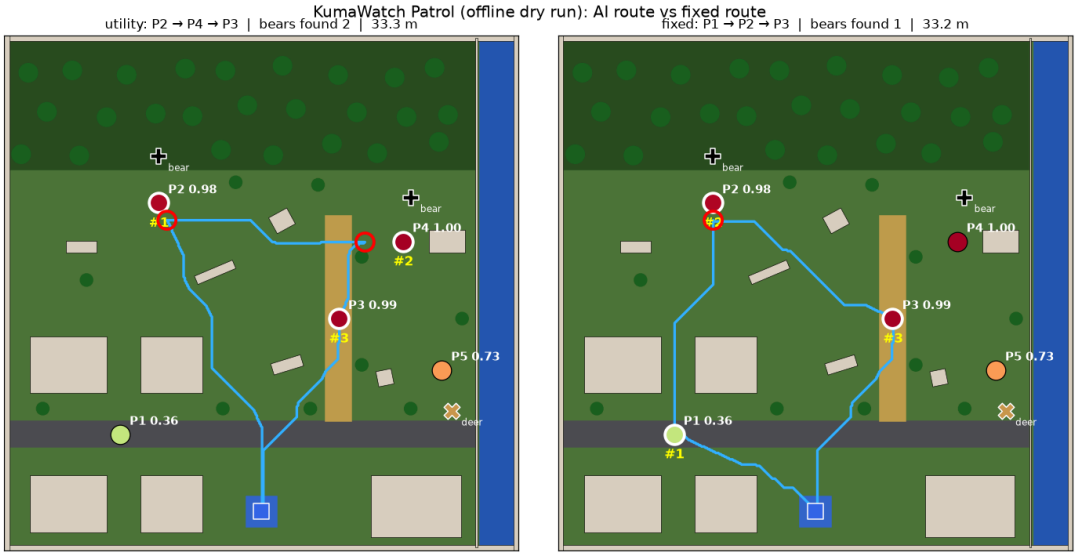


図3　巡回の比較（俯瞰図）。左：AI 巡回（P2→P4→P3）、右：固定巡回（P1→P2→P3）。 円の色はリスク、白枠は選ばれた地点、＋は熊、×はシカ、赤丸は熊の発見位置。

指標	固定巡回（P1→P2→P3）	AI 巡回（P2→P4→P3）
巡回地点数	3	3
熊の発見数	1	**2**
移動距離（Nav2 による走行）	28.8 m	32.5 m
所要時間（シミュレーション時間）	152.6 s	153.8 s

表3　同じ 3 地点・同じ熊の配置での比較（詰所への帰還を含む）。本物の Nav2（AMCL・NavFn・DWB）と本作品の地図・パラメータを、 差動二輪とレイキャスト LiDAR の簡易シミュレータ上で動かした値で、熊は既知座標で判定した。

同じ 3 地点を回る条件で、AI 巡回は熊を 2 頭、固定巡回は 1 頭見つけた。移動距離は AI 巡回の方が 3.7 m 長いが、 所要時間はほぼ同じだった（AI 巡回は熊の手前で止まって次へ向かうため）。ただし、これは 1 日・1 配置のシミュレーションの結果であり、 「AI が一般に優れている」ことを示すものではない。実際、1 年を通した評価では、山形県で TTM は単純な静的予測（過去の平均）に有意に劣っている。

6.2　熊判定の精度

0.94
Precision（検証 105 枚）

0.99
Recall（検証 105 枚）

0.985
mAP@0.5

11 / 11
巡回中の視点での判定

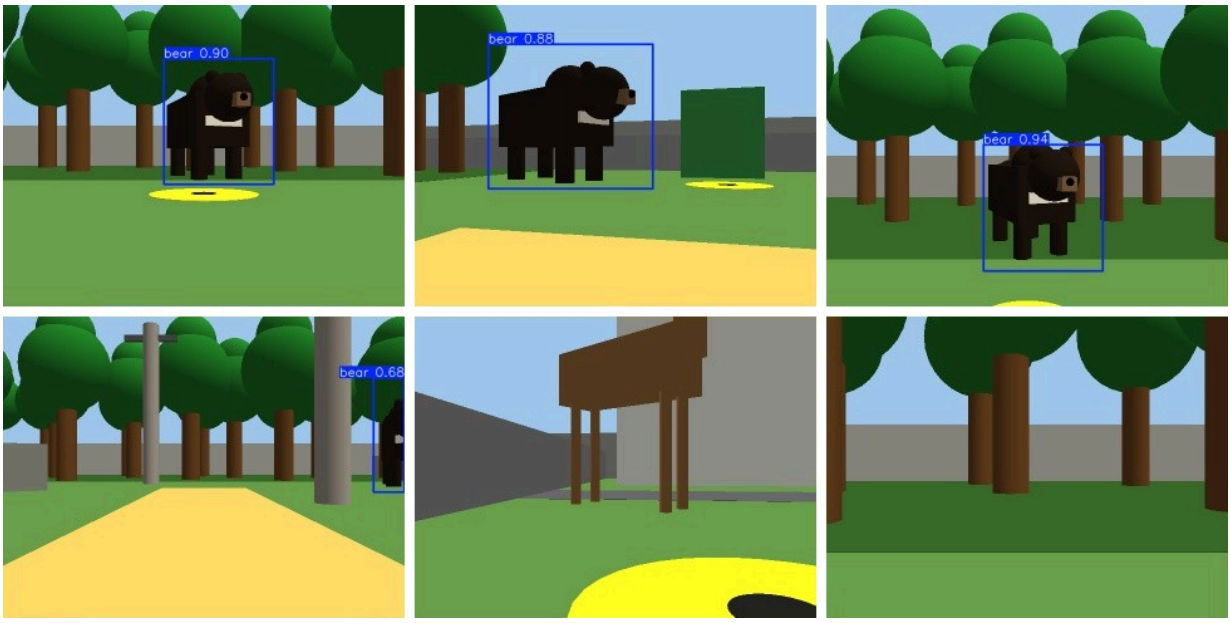


図4　巡回中にロボットが見る視点での判定例。上段：P2 への接近（信頼度 0.90）、P4 への接近（0.88）、 カメラの高い Happy Mini の視点（0.94）。下段：P3 から見た画面の端の熊（5.4 m 先で停止距離の外）、 シカ（熊と誤認しない）、熊のいない林縁。

巡回中の 11 視点（P2・P4 への接近と到着、熊のいない P1・P3・P5、シカ、林縁など）では、熊が写る視点はすべて検出し（信頼度 0.88〜0.98）、 シカ・木・建物を熊と誤認しなかった。ROS 2 ノードとしても、Gazebo のカメラ映像から熊を検出し、停止指示のメッセージを出すことを確認した。

7 限界と今後の課題

- **シミュレーションの範囲。**Gazebo 内の模擬環境であり、実環境での自己位置推定・通信・天候の影響は検証していない。
- **熊判定は Gazebo の熊でのみ学習・評価した。**基本図形の熊と実物の熊は見た目が大きく違うため、実物の画像での精度は不明である。 実運用には実画像での学習と評価が必要である。
- **予測と地点の対応は説明用。**TTM の予測は 10 km 四方のセル単位で、町の中の 1 地点と同じ意味ではない。日付もシナリオのために選んだ 1 日である。
- **今後。**熊検知後の安全地点への退避と発見地点の地図表示、GLM-Logit や静的予測との巡回結果の比較、熊の配置を変えた複数回の試行による発見率の評価を行う。

付録 動作環境・再現方法・使用した素材

動作環境：Ubuntu 22.04、ROS 2 Humble、Gazebo Classic 11、Navigation2、TurtleBot3（waffle / Happy Mini）、Ultralytics YOLOv8。 講義の演習環境（Docker イメージ `airobotbook/ros2-desktop-ai-robot-book-humble`）で動作する。 起動：`ros2 launch kumawatch_mission kumawatch_patrol.launch.py`（固定巡回は `strategy:=fixed`）。 詳しい手順はリポジトリの `README.md` と `docs/lecture_env.md` を参照。学習データと学習プログラムは `training/` にまとめた。

- KumaWatch（熊出没予測の研究。TTM の予測値。データは CC BY 4.0、出典：山形県の公開野生動物出没データ）：github.com/todalaba/KumaWatch
- IBM Granite TinyTimeMixer（Apache-2.0）、Ultralytics YOLOv8（AGPL-3.0）
- ROS 2 / Navigation2 / Gazebo（Apache-2.0）、TurtleBot3 シミュレーション（ROBOTIS, Apache-2.0）、Happy Mini（AI ロボット入門）
- 熊・シカ・建物・樹木のモデルは基本図形から自作（外部の 3D モデルは不使用）